

Deep Monkey Studio: An Integrated Platform for Industrial Digital Twin Authoring, Execution, and Delivery

Wenpeng Zhang
Independent Researcher
15184552744@163.com

Working draft — 9 October 2026

Code: <https://github.com/fatasia/DeepMonkey-Studio>

Study status. Implementation study with recorded rendering measurements and targeted software tests. Comparative lifecycle experiments remain to be completed.

Abstract

Industrial digital twins combine heterogeneous engineering models, live device data, interactive graphics, and operational logic. Maintaining the identity and state of an asset across these layers becomes difficult when models change, users and agents edit concurrently, or execution is interrupted. Deep Monkey Studio integrates model ingestion, industrial connectivity, 2D/3D authoring, a WebGPU rendering engine, ontology-bound actions, physical simulation, and native delivery. Its architecture connects revisioned assets, validated scene transactions, persistent agent checkpoints, and executable publication artifacts. We describe the implemented platform and evaluate selected mechanisms through 86 targeted software tests. An archived rendering comparison also exposes a concrete tradeoff: the Deep WebGPU backend reports seven draw calls for a 1,000-object fixture, while first-frame latency and dynamic frame pacing remain optimization targets. We formulate four lifecycle experiments to measure identity preservation, editing correctness, recovery, and cross-runtime consistency. The paper’s central question is whether integration through explicit identities and revision checks reduces industrial workflow failures and integration effort.

1 Introduction

An industrial digital twin must remain useful after its first demonstration. A revised CAD assembly should retain device bindings; an incoming measurement should update the intended asset; a maintenance action should preserve unrelated edits; and a delivered application should represent the approved scene. These requirements span graphics, data integration, semantics, execution, and deployment.

Deep Monkey Studio addresses this workflow as one platform. The authoring environment combines scene editing, dashboards, scripts, plugins, and industrial agents. Shared contracts connect these tools to model conversion, device data, simulation, and browser/native runtimes. The engineering objective is to make changes traceable across the full asset lifecycle while keeping the system extensible and locally deployable.

The system design has three parts: a modular rendering and execution engine with explicit resource and shader contracts; a programmable authoring platform connecting engineering assets, device data, editors, and domain actions; and revisioned scene artifacts connecting authoring to

browser and native delivery. We describe their interfaces and design choices, then examine the available software and rendering evidence. Battery prediction algorithms are studied in a companion paper; here they illustrate a domain runtime that consumes platform contracts.

Research questions. RQ1: How well are asset identity and user overrides preserved through import, revision, editing, and delivery? RQ2: Do revision checks and semantic action admission reduce incorrect edits under concurrent human and agent activity? RQ3: What recovery, rendering, and integration costs accompany these mechanisms?

2 Related Work

Holodeck studies language-guided generation of embodied environments through assets and spatial constraints [1]. SceneCraft combines structured scene generation with procedural tools [2], while RoboGen connects generated tasks and environments to robotic skill learning [3]. Code2Games connects scene and gameplay representations with engine and behavior feedback [4]. These works inform our use of explicit representations and executable evaluation.

Existing digital-twin software also extends beyond visualization demos. Eclipse Ditto provides device-state middleware [5]; iTwin.js provides infrastructure-twin development libraries [6]; OpenTwins combines 3D visualization, IoT streams, and machine-learning predictions [7]. Our comparison therefore concerns workflow coverage, dependency requirements, state preservation, and measured operating cost. The existence of an integrated implementation alone does not establish superiority over these systems.

3 Platform Architecture

Layer	Responsibilities and interfaces
Inputs	Engineering files, device protocols, database queries, event streams; source identity and format profile.
Asset and data	Revised model packages, hierarchy and properties, normalized data rows and events, object bindings.
Authoring	2D/3D editors, dashboards, scripts, plugins, scene transactions, human and agent tools.
Execution	WebGPU rendering, ontology actions, persistent agent runs, headless physics, domain services.
Delivery	Compiled scene artifacts, runtime packages, browser/native consumers, content hashes and runtime evidence.

The implementation combines TypeScript/React application code with Rust/native components. Its existing Three.js viewer and Deep WebGPU backend coexist. Engine-neutral identities and versioned packages connect authoring to execution; each adapter has an explicit supported profile.

3.1 Deep Engine: Rendering, Resources, and Cross-Runtime Execution

Deep Engine provides resource packets, instancing, render scheduling, shader packages, and WebGPU execution. The rendering implementation includes GGX materials, image-based lighting, directional shadows, linear HDR, tone mapping, transparency, and postprocessing. Its glTF path

handles supported texture channels, node animation, skinning, and morph targets. Unsupported encoding features produce structured errors.

Resource residency shares a geometry/texture budget. Visible and prefetched chunks request levels of detail; candidates are validated before publication at a frame boundary. Cancellation and superseded requests govern pending uploads and cleanup. Shader packages carry entry points, bindings, vertex layouts, and pass configuration, with shared TypeScript/Rust representations. A versioned 2D display list carries paths, text, images, clipping, and hit regions into native chart rendering.

These mechanisms support local delivery and backend evolution. Product-wide visual equivalence, device-loss recovery, and complete animation/physics coverage require separate acceptance runs from isolated rendering benchmarks.

Scheduling and resource reuse. The render graph validates dependencies and orders passes topologically, producing temporary-resource lifetimes and physical-slot plans. Aliasing requires compatible format, dimensions, sample count, and usage; external resources are excluded from temporary slots. This makes reuse an explicit scheduling decision. For streamed scenes, residency planning retains the coarsest drawable level for visible objects before adding geometry detail and texture mips within the budget. Ownership and cleanup rules handle superseded uploads.

Programmable materials and execution interfaces. The DeepSL surface compilation path translates Standard/Unlit material properties into typed intermediate representations and WGSL. Versioned shader packages specify entry points, bindings, vertex streams, attachments, and render state for validation and pipeline creation by browser and Rust executors. Caches distinguish persistent content from device-specific objects. Scene state is separated from the backend, while the host controls drawing and backend handover. These interfaces serve editor integration, standalone SDK use, and native playback paths.

3.2 Authoring Coverage and Extensibility

The platform organizes 2D pages, 3D scenes, data bindings, scripts, and plugins within a project. Model hierarchy and properties support object discovery and selection; device data supplies bound-object state; dashboards present project data; domain services provide computation for actions and analysis. The engine supplies visual and runtime execution, while editors provide interaction. Scene transactions and publication artifacts connect editing to deployment. This separation supports both modular engine use and an integrated authoring application.

Functional coverage is organized around a task path: ingest engineering models and device data, bind objects, edit scenes and pages, execute domain actions, save, and deliver. Format and protocol breadth determine input coverage; scripts, plugins, and SDKs determine extension points; runtime packages and local deployment determine delivery options. The following sections describe the inspected input paths, operations, and measured scope.

3.3 Model Ingestion and Fidelity

Imports retain source provenance and produce runtime geometry with associated hierarchy and properties. Format coverage is described by version/profile and retained information. Table 1 summarizes the inspected implementation paths and documentation; it is not an exhaustive compatibility certification.

Table 1: Engineering model paths and their scope.

Format	Implemented path or tested scope
glTF / GLB	Runtime exchange, optimization and publication; loader-specific extension coverage.
IFC	BIM ingestion through fragments, preserving a source representation and semantic access.
STEP / STP	Open-source tessellation into runtime geometry, with hierarchy/property sidecars.
FBX; DXF	Scene/animation loading and 2D drawing ingestion through their respective adapters.
OpenUSD	USDA/USDC/USDZ loading with inspected sample coverage; complex composition remains a separate validation target.
JT	Built-in LOD0 triangle geometry for inspected 9.5/10.3 profiles, assembly instances and partial properties/materials; PMI structure is distinct from semantic annotations.
X_T	Built-in text parsing for controlled geometric subsets. Trim, surface-family, naming, color, and assembly losses depend on the profile.

The broader catalog also records candidate formats. A catalog entry is not evidence of a working importer. Native proprietary formats require an admitted built-in/open-source profile before inclusion in the operational coverage matrix. Evaluation must score geometry, assembly structure, properties, interaction, and delivery independently.

3.4 Industrial Protocols and Data Sources

The API dispatches real connector adapters and normalizes returned data for consumers. Continuous subscriptions and bounded previews serve different operational needs.

Interface family	Inspected operations
MQTT; OPC UA	Persistent subscriptions and preview reads. Shared subscription lifecycle; OPC UA quality, timestamps, and configured security handling.
Modbus TCP; BACnet; S7; EtherNet/IP	Bounded reads of registers/coils, properties, PLC addresses, or tags.
HTTP; WebSocket; Kafka; AMQP	Request or bounded message sampling through source-specific adapters.
SNMP; CoAP; TCP; UDP; serial	Source-specific sampling adapters.
SQL and analytical stores	Query adapters including PostgreSQL, MySQL-compatible stores, SQL Server, Oracle, ClickHouse, and time-series sources.
Document and file sources	MongoDB, Elasticsearch, CSV, and Excel adapters.

Persistent MQTT and OPC UA sources feed the event infrastructure. Simulation-result export includes MQTT and SQL paths. Actual device coverage depends on transport settings, address

mapping, data types, and vendor behavior; a field evaluation should record these alongside reconnection and event-ordering outcomes.

3.5 Asset Revision and Controlled Editing

Source hashes and package identities associate a revision with its asset. The reimport coordinator prepares updates before publication and supports rollback, generation checks, and conflict handling. The product adapter updates scene snapshots while retaining identity relationships and applicable overrides.

Scene transactions carry a base revision and validated commands. The transaction layer rejects stale revisions and invokes rollback after partial execution, errors, or cancellation. Editor and agent consumers use this path; the API/browser bridge preserves transaction identity and caches outcomes. This gives concurrent work a concrete admission boundary.

3.6 Ontology, Agents, and Simulation

Ontology actions bind canonical targets to published definitions, arguments, scope, and preconditions. Execution produces a receipt. Property constraints have report-only and strict behavior, so the selected mode belongs in each experiment.

The connected industrial agent runtime uses a tool gateway, persisted checkpoints, cancellation, and revision-aware recovery. Its Web workspace exposes resume and cancel operations. A separate task-graph helper supports bounded correction but has no identified product consumer in this audit.

The World API provides reset, step, observe, snapshot, restore, and close operations over a Rapier-based headless runtime. The inspected profile runs at 60 Hz, with at most 256 objects and 600 ticks per step. Observations include poses, contacts, and events; camera/depth/lidar sampling is outside this profile. Rigid visual models require supported collider declarations. Restore checks snapshot integrity and replaces a session only after successful reconstruction.

4 Evaluation

4.1 Targeted Software Verification

The 9 October 2026 verification run passed 86 tests across nine files: 40 world-runtime tests, 37 API tests covering ontology actions, agent stop/recovery, the World API and editor transaction bridge, five scene-transaction tests, and four task-graph tests. These tests exercise selected admission, replay, ownership, rollback, and recovery behavior. The source review also traced the corresponding product consumers.

4.2 Recorded Rendering Comparison

The 7 October benchmark used three cold contexts per configuration on an i9-12900HX and RTX 4060 Laptop GPU, at 1440×900, DPR 1, with PBR, ACES, and 1024-resolution shadows. Table 2 selects the 1,000-object, 614,294-triangle fixture. Frame P95 is the median across runs of browser RAF measurements; GPU P95 uses timestamp measurements. Deep WebGPU combines a low reported draw count with competitive static frame pacing in this fixture. Its first frame is substantially later, and dynamic frame pacing is worse than the Three WebGL reference. These results identify startup and update costs for further optimization. They do not resolve full-product visual parity or backend promotion.

Backend	Workload	First frame ms	Frame P95 ms	GPU P95 ms	Draws
Three WebGL	Static	72.7	7.1	4.2	1055
Three WebGPU	Static	283.2	7.1	1.6	1056
Babylon WebGPU	Static	876.0	15.0	4.5	2001
Deep WebGPU	Static	1623.3	7.1	3.1	7
Three WebGL	Dynamic	70.4	7.1	4.1	1055
Deep WebGPU	Dynamic	1733.7	14.0	3.1	7

Table 2: Archived isolated rendering measurements. Draw counters are engine-specific and not fully equivalent.

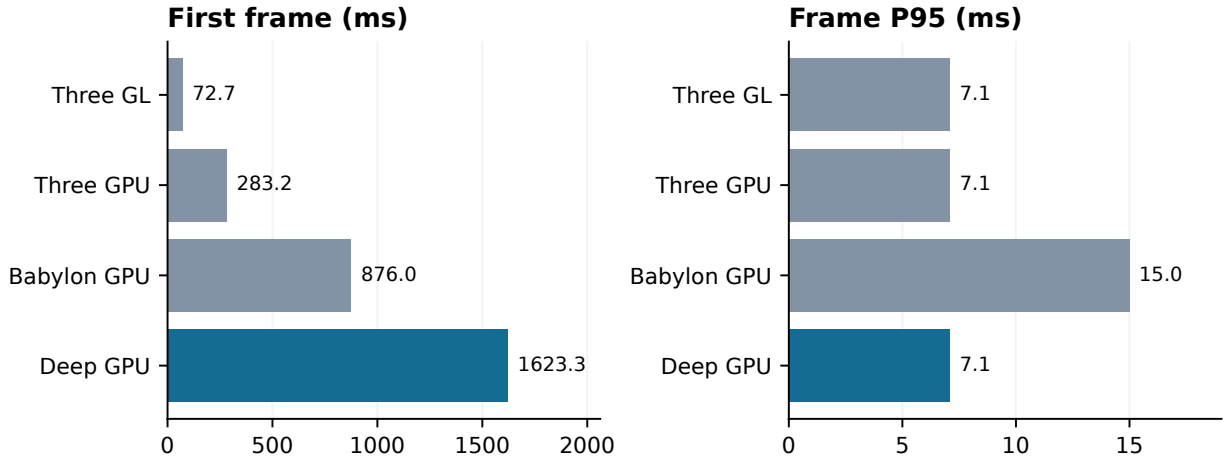


Figure 1: Static 1,000-object fixture from the archived comparison. Startup and steady-state frame pacing expose different costs; lower is better.

4.3 Lifecycle Experiments to Complete

Four experiments connect the platform layers:

1. **Model maintenance:** import a supported real assembly, bind devices, revise source geometry and names, preserve overrides, save/reopen, and deliver. Measure identity retention, broken references, fidelity, and completion time.
2. **Concurrent repair:** edit the scene between an agent’s plan and commit. Measure stale-write rejection, correct-target actions, collateral edits, and successful replanning.
3. **Interrupted execution:** cancel/resume agent work and restore physical snapshots. Measure lost or duplicated actions, recovery time, and assertion agreement.
4. **Cross-runtime delivery:** publish the same scene to browser and native runtimes. Measure transforms, materials, identity, image differences, startup, memory, and frame latency.

Freeze assets, expected mappings, software versions, and failure injections before running comparisons. Use identical model/tool budgets for tool-connected agents with and without the selected identity/revision mechanisms. Keep task families as the independent analysis unit; report failed

tasks and paired uncertainty. Compare an independently assembled workflow on the same requirements to measure integration effort. This experiment needs an explicit dependency list and reproducible setup time, rather than an assumed advantage from fewer applications.

5 Discussion and Limitations

The platform’s practical value lies in editable source, local runtime paths, common identities, and inspection of state changes across tools. These properties make customization and maintenance possible within the applicable license. Commercial readiness additionally depends on sustained field workloads, upgrade behavior, operator usability, and support costs; those measurements are not yet available in this study.

Current evidence covers selected software behaviors and isolated rendering. Sequential receipt replay does not establish concurrent exactly-once execution, and transaction atomicity depends on the driver. Format fixtures do not cover all versions or geometries. A complete lifecycle comparison is the main remaining empirical task.

6 Reproducibility and Availability

The platform source is available at <https://github.com/fatasia/DeepMonkey-Studio>. Commit [8bbfd90535db](#) identifies the implementation inspected for this update; historical rendering measurements retain their own recorded revisions. The [Deep Engine documentation](#) describes engine modules and supported profiles, and the [deployment guide](#) provides startup instructions.

The accompanying evidence notes identify source files, reviewed artifacts, test commands, and remaining experiments. Rendering records use their own recorded revision and hardware; manuscript tables retain that distinction from the current implementation audit. A reproducible release must preserve asset bytes, profile declarations, dependency/checkpoint versions, action traces, and expected outcomes.

Deep Monkey Studio is source-available under a modified MIT license with additional corporate conduct and human-rights provisions, named the Deep Monkey Community Source License 1.0 (DMCSL-1.0). The repository license specifies the applicable terms. Article and third-party asset licenses are separate.

Generative AI assisted source inspection, organization, and manuscript drafting.

7 Conclusion

Deep Monkey Studio integrates industrial model ingestion, connectivity, interactive authoring, rendering, agents, simulation, and delivery. Revisioned assets and controlled state changes provide the architectural connection between these capabilities. Existing tests support selected mechanisms, and rendering measurements expose useful performance tradeoffs. The next evaluation measures whether those mechanisms preserve industrial task correctness through change, interruption, and deployment.

References

- [1] Y. Yang et al. Holodeck: Language Guided Generation of 3D Embodied AI Environments. CVPR, 2024. <https://arxiv.org/abs/2312.09067>.

- [2] Z. Hu et al. SceneCraft: An LLM Agent for Synthesizing 3D Scenes as Blender Code. ICML, 2024. <https://proceedings.mlr.press/v235/hu24g.html>.
- [3] Y. Wang et al. RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation. ICML, 2024. <https://proceedings.mlr.press/v235/wang24cc.html>.
- [4] W. Wu et al. Code2Games: Enabling Coding Agents for Gaming World Generation. arXiv:2610.05033, 2026. <https://arxiv.org/abs/2610.05033>.
- [5] Eclipse Foundation. Eclipse Ditto: Overview. Accessed 9 October 2026. <https://eclipse.dev/ditto/intro-overview.html>.
- [6] iTwin.js. Getting Started. Accessed 9 October 2026. <https://www.itwinjs.org/learning/>.
- [7] J. Robles, C. Martín, and M. Díaz. OpenTwins: An open-source framework for the design, development and integration of effective 3D-IoT-AI-powered digital twins. arXiv:2301.05560, 2023. <https://arxiv.org/abs/2301.05560>.